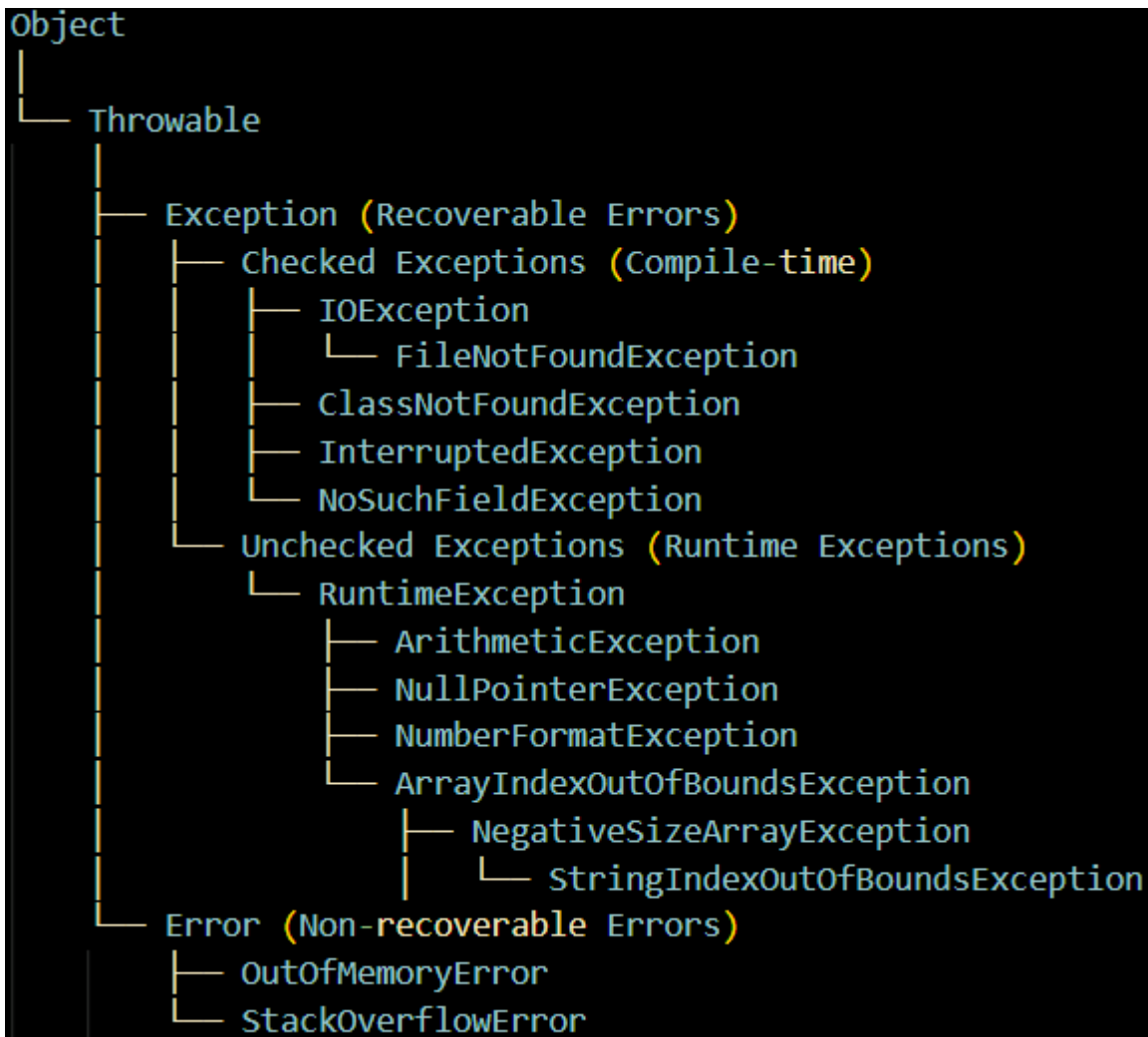


MODULE 1

Feature	static	final	abstract
Meaning	Belongs to the class	Cannot be modified	Defines incomplete methods/classes
Usage	Variables, methods, blocks	Variables, methods, classes	Methods and classes
Inheritance	Not inherited (shared)	Cannot be overridden/inherited	Must be inherited and implemented
Object Requirement	No object required	Object required	Cannot instantiate abstract classes
Method Body	Must have a body	Must have a body	No body (for abstract methods)
Combination	Cannot be abstract	Cannot be abstract	Cannot be static or final
Example	<code>static int x = 5;</code>	<code>final int y = 10;</code>	<code>abstract void display();</code>

Feature	Method Overloading	Method Overriding
Definition	Defining multiple methods with the same name but different parameters (within the same class).	Redefining a method from the parent class in a subclass to provide a specific implementation.
Class Relationship	Happens within the same class .	Happens between superclass and subclass (inheritance required).
Method Signature	Must differ in number, type, or order of parameters.	Must have the same method signature (name, parameters, and return type).
Access Modifier	No restriction (can have any access level).	Cannot reduce visibility (e.g., <code>public</code> cannot be overridden as <code>private</code>).
Return Type	Can be different for overloaded methods.	Must be the same or covariant (subtype of parent method's return type).
Static/Final Methods	Can overload <code>static</code> and <code>final</code> methods.	Cannot override <code>static</code> or <code>final</code> methods.
Polymorphism Type	Compile-time polymorphism (decision made at compile-time).	Runtime polymorphism (decision made at runtime using dynamic method dispatch).
Usage	Used when methods perform similar actions with different inputs .	Used to customize or extend behavior of an inherited method.
Annotations	No annotation required. 	Typically uses <code>@Override</code> annotation (optional but recommended).
Constructor	Constructors can be overloaded.	Constructors cannot be overridden.
Example	<pre>java class A { void add(int a, int b) {} void add(double a, double b) {} }</pre>	<pre>java class A { void show() {} } class B extends A { @Override void show() {} }</pre>

MODULE 2



Exception	Category	Meaning
IOException	Checked Exception	General input-output errors.
FileNotFoundException	Checked Exception	Accessing a non-existent file.
ClassNotFoundException	Checked Exception	Class is not found during runtime.
InterruptedException	Checked Exception	A thread is interrupted during execution.
NoSuchFieldException	Checked Exception	Accessing a field that does not exist.
RuntimeException	Unchecked Exception	Parent class for runtime errors.
ArithmeticException	Unchecked Exception	Invalid arithmetic operation (e.g., divide by zero).
NullPointerException	Unchecked Exception	Accessing methods/fields on a null object.
NumberFormatException	Unchecked Exception	Invalid conversion of a string to a number.
ArrayIndexOutOfBoundsException	Unchecked Exception	Accessing an invalid array index.
NegativeSizeArrayException	Unchecked Exception	Creating an array with a negative size.
StringIndexOutOfBoundsException	Unchecked Exception	Invalid index access in a string.

MODULE 5

What is Maven?

- **Maven** is a **build automation** and **project management tool** developed by the **Apache Software Foundation**.
 - **Maven** is used to **build the code** and manage **dependencies** (libraries like JAR files).
 - In traditional Java projects, libraries like Spring or Hibernate must be **downloaded and added manually**. If the version changes, you have to redo everything.
 - **Maven automates** this by allowing you to **declare dependencies** in a file called pom.xml, and it takes care of downloading and managing them.
-

Key Features of Maven

- **Dependency Management:** Just write the needed dependencies in pom.xml, and Maven will download and manage them.
 - **Build Automation:** Maven handles tasks like compiling code, packaging, testing, etc.
 - **Project Management:** It provides a standardized project structure and lifecycle.
-

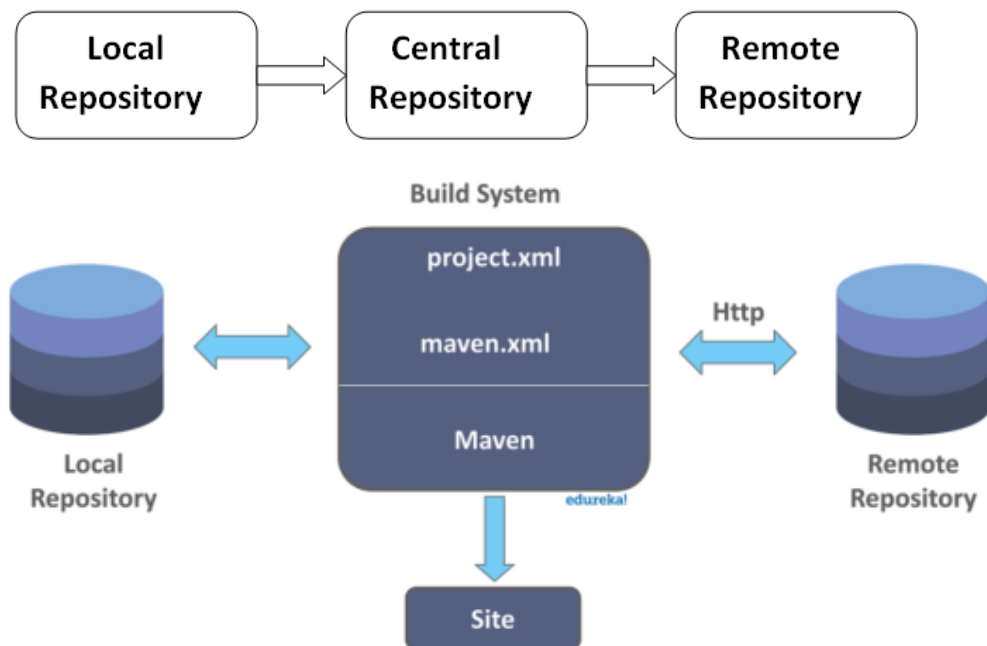
How Maven Works

- It uses **conventions**, meaning you don't need to specify everything—just the exceptions.
 - The **POM file** (Project Object Model, pom.xml) describes:
 - The project itself
 - Its dependencies
 - Build order
 - Plugins and directories used
 - Command to check if maven has been successfully installed and added to the path – mvn -version
-

Maven Architecture

A maven repository is a directory of packaged JAR file with pom.xml file. Maven searches for dependencies in the repositories. There are 3 types of maven repository:

- Maven **local repository** is located in your local system. It is created by the maven when you run any maven command.
- Maven **central repository** is located on the web. It has been created by the apache maven community itself.
- Maven **remote repository** is located on the web. Most of libraries can be missing from the central repository such as JBoss library etc, so we need to define remote repository in pom.xml file.



Maven Built-In Build Life Cycles

There are 3 main **life cycles**:

1. **Clean** – Cleans up previous builds. It has 3 phases. 3 commands : mvn pre-clean, mvn clean, mvn post-clean
2. **Default** – This phase handles the complete deployment of the project. It has 23 Phases. It is the main build process (compile, test, package, install).
3. **Site** – Creates project documentation. It has 4 phases: pre-site, site, post-site, site-deploy.

The command used in maven to generate javadocs for a given project is 'mvn site'. Basically, when this command is invoked, maven calls 'Doxia' document generation and other report generating plugins.

Doxia is basically a framework used for content generation by maven. This generates contents both in static and dynamic ways.

- A lifecycle is organized into sequence of phases.
- Each phase is responsible for a specific task.
- Phases give order to a set of goals.
- These goals are chosen and bound by the packaging type of the project being acted on.

MAIN PHASES OF DEFAULT LIFE CYCLE

• **Validate**

- Checks if all the information necessary for the build is available;

• **Compile**

- Compiles the source code of the project

• **Test compile**

- Compiles the test source code
- Responsible for running the unit test of the application

• **Packaging**

- Packaging the compiled source code into the distributable format (i.e jar format)

• **Integration**

- In this phase the process and deployment of the package is needed

• **Install**

- Install the package to a local repository

• **Deploy**

- Copy the package to a remote repository

Maven Coordinates (GAV)

These uniquely identify any dependency:

- **project** – It is the root element of pom.xml file.
- **groupId** – Who made the project (e.g., org.springframework)
- **artifactId** – The name of the library (e.g., spring-core)
- **version** – The version of the library (e.g., 4.3.8.RELEASE)
- **modelVersion** – Tells Maven how to read the pom.xml file

Additional Elements:

- **packaging** – defines packaging type such as jar, war etc.
- **name** – defines name of the maven project.
- **url** – defines url of the project.
- **dependencies** – defines dependencies for this project.
- **dependency** – defines a dependency. It is used inside dependencies.
- **scope** – defines scope for this maven project. It can be compile, provided, runtime, test and system.

POM & Super POM

- **POM (Project Object Model)**: It's the heart of any Maven project, stored in pom.xml.
 - **Super POM**: A default POM provided by Maven that your project inherits from. It sets some default behaviors.
-